

Q1. Launch a t3.nano Ubuntu 20.04 EC2 instance in a public subnet, SSH into it with your own key pair, make at least one visible configuration or software change, then create an AMI from this instance, terminate it, and launch a new instance from that AMI to verify the change persists.

Ans :

i-042f93173703938a5 (MYEC2)		
Details Status and alarms Monitoring Security Networking Storage Tags		
▼ Instance summary Info		
Instance ID i-042f93173703938a5	Public IPv4 address 13.62.18.85 open address	Private IPv4 addresses 172.31.28.164
IPv6 address -	Instance state Running	Public DNS ec2-13-62-18-85.eu-north-1.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-28-164.eu-north-1.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-28-164.eu-north-1.compute.internal	Elastic IP addresses -
Answer private resource DNS name -	Instance type t3.micro	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 13.62.18.85 [Public IP]	VPC ID vpc-030a25f19041964b6	Auto Scaling Group name -
IAM Role -	Subnet ID subnet-016a7ab4a21cf97e5	

This is my EC2 instance created on Ubuntu 24.04 because the 20.04 version is now deprecated now .

Now configuring the change in my EC2 instance to install nginx

```
ubuntu@ip-172-31-28-164: ~  
  
ubuntu@ip-172-31-28-164:~$ nginx  
Command 'nginx' not found, but can be installed with:  
sudo apt install nginx  
ubuntu@ip-172-31-28-164:~$ sudo apt install nginx  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  nginx-common  
Suggested packages:  
  fcgiwrap nginx-doc ssl-cert  
The following NEW packages will be installed:  
  nginx nginx-common  
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.  
Need to get 564 kB of archives.  
After this operation, 1596 kB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 n  
ginx-common all 1.24.0-2ubuntu7.5 [43.4 kB]  
Get:2 http://eu-north-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 n  
ginx amd64 1.24.0-2ubuntu7.5 [520 kB]  
Fetched 564 kB in 0s (16.6 MB/s)  
Preconfiguring packages
```

The changes are reflected as ,

```

ubuntu@ip-172-31-28-164:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: en>
   Active: active (running) since Thu 2026-01-29 09:03:39 UTC; 1min 20s ago
     Docs: man:nginx(8)
  Process: 1398 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_proce>
  Process: 1400 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (c>
 Main PID: 1430 (nginx)
    Tasks: 3 (limit: 1008)
   Memory: 2.4M (peak: 5.3M)
      CPU: 28ms
   CGroup: /system.slice/nginx.service
           └─1430 "nginx: master process /usr/sbin/nginx -g daemon on; master>
             └─1432 "nginx: worker process"
               └─1433 "nginx: worker process"

Jan 29 09:03:39 ip-172-31-28-164 systemd[1]: Starting nginx.service - A high pe>
Jan 29 09:03:39 ip-172-31-28-164 systemd[1]: Started nginx.service - A high per>
lines 1-17/17 (END)

```

The AMI created ,

AMI ID: ami-053bc6e18c2116215			
Details Permissions Storage My AMI usage AMI ancestry - new Tags			
AMI ID ami-053bc6e18c2116215	Image type machine	Platform details Linux/UNIX	Root device type EBS
AMI name MyAMI	Owner account ID 797265029671	Architecture x86_64	Usage operation RunInstances
Root device name /dev/sda1	Status Pending	Source 797265029671/MyAMI	Virtualization type hvm
Boot mode uefi-preferred	State reason -	Creation date 2026-01-29T09:10:52.000Z	Kernel ID -
Description EC2 Image with NGINX installed	Product codes -	RAM disk ID -	Deprecation time -

The instance created from the AMI and it shows the configuration which the base EC2 had already.

```

ubuntu@ip-172-31-14-142:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: en>
   Active: active (running) since Thu 2026-01-29 09:14:11 UTC; 1min 24s ago
     Docs: man:nginx(8)
  Process: 587 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_proces>
  Process: 647 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (co>
 Main PID: 658 (nginx)
    Tasks: 3 (limit: 1017)
   Memory: 3.7M (peak: 3.8M)
      CPU: 26ms
   CGroup: /system.slice/nginx.service
           └─658 "nginx: master process /usr/sbin/nginx -g daemon on; master_>
             └─660 "nginx: worker process"
               └─661 "nginx: worker process"

Jan 29 09:14:10 ip-172-31-14-142 systemd[1]: Starting nginx.service - A high pe>
Jan 29 09:14:11 ip-172-31-14-142 systemd[1]: Started nginx.service - A high per>
lines 1-17/17 (END)

```

Q2. For a memory-heavy, CPU-light internal dashboard that also runs Redis on the same instance, compare at least three suitable EC2 instance types from the console/pricing

page and then actually launch the one you choose, deploy a simple workload to generate CPU and memory usage, and validate on the instance that the resources match your expectations.

Ans :

Since we need memory-heavy CPU to handle heavy workload so I checked three instances for comparing :

1. .r6 instance : It works for heavy memory workloads with providing support for SQL , NOSQL databases offering 3.5GHz intel xeon processors , 128vCPUs ,and upto 200 gbps network bandwidth , with 0.1008 usd per hour usage on Linux base pricing.
2. .r7i instance : It works much better when compared with above instance suitable for all SQL , NOSQL workloads and memory intensive workloads with 3.2ghz 4th gen processors ,192vCPUs and 0.1071 usd per hour usage on Linux base pricing.
3. .r8g instance : It offers best price performance in AWS ec2 and can handle extensive memory workloads such big data analytics , open source databases etc. It provides custom build AWS graviton processors and offers 40gbps amazon elastic block storage charging 0.11782 usd per hour usage on Linux base system.

So based on the above the three choices I prefer to go with r7 instance because it offers best pricing and our required services requirement will be fulfilled easily with the instance.

The screenshot displays the AWS Management Console interface for EC2 instances. At the top, there's a search bar and a filter for 'All states'. Below this, a table lists the instances. The first instance, 'RedisEC2' with ID 'i-035bc3bf029d05713', is in a 'Running' state, using the 'r7a.large' instance type, and is located in the 'us-east-1b' availability zone. Its public IP address is '98.94.33.77'. Below the table, the details for the selected instance 'i-035bc3bf029d05713 (RedisEC2)' are shown. The 'Instance summary' section includes the instance ID, IPv6 address (none), hostname type (ip-172-31-4-6.ec2.internal), and answer private resource DNS name (none). The 'Public IPv4 address' is '98.94.33.77', and the 'Private IP DNS name (IPv4 only)' is 'ip-172-31-4-6.ec2.internal'. The 'Instance type' is 'r7a.large'. The 'Private IPv4 addresses' section shows '172.31.4.6', and the 'Public DNS' section shows 'ec2-98-94-33-77.compute-1.amazonaws.com'. The 'Elastic IP addresses' section is empty.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP
RedisEC2	i-035bc3bf029d05713	Running	r7a.large	Initializing	View alarms +	us-east-1b	ec2-98-94-33-77.comp...	98.94.33.77	-

i-035bc3bf029d05713 (RedisEC2)

Instance summary

Instance ID: i-035bc3bf029d05713

IPv6 address: -

Hostname type: IP name: ip-172-31-4-6.ec2.internal

Answer private resource DNS name: -

Public IPv4 address: 98.94.33.77 | open address

Instance state: Running

Private IP DNS name (IPv4 only): ip-172-31-4-6.ec2.internal

Instance type: r7a.large

Private IPv4 addresses: 172.31.4.6

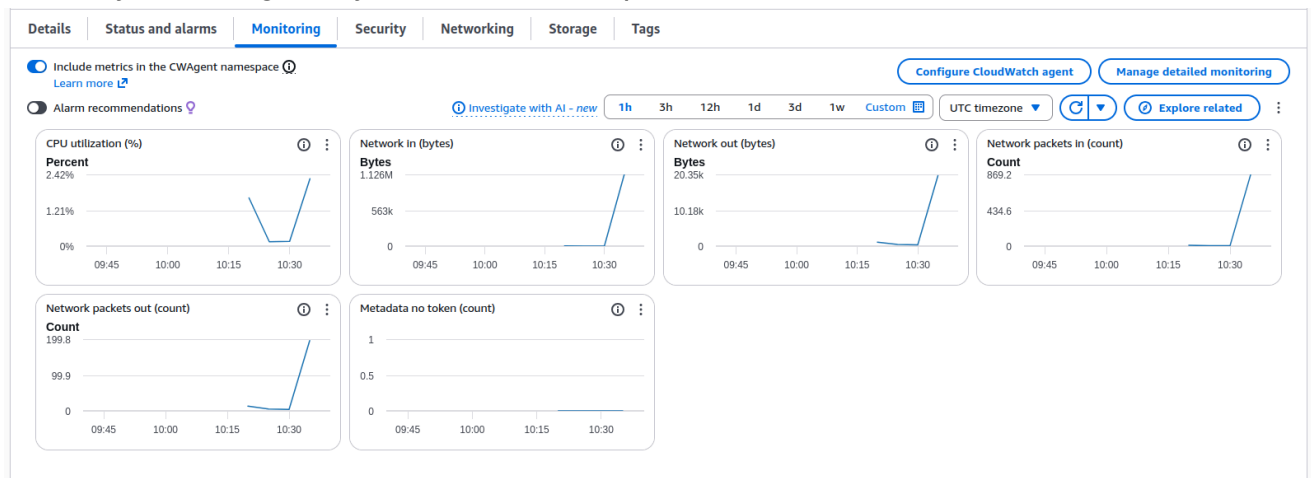
Public DNS: ec2-98-94-33-77.compute-1.amazonaws.com | open address

Elastic IP addresses: -

My EC2 with the chosen configuration.

```
ec2-user@ip-172-31-4-6:~  
[ec2-user@ip-172-31-4-6 ~]$ free -h  
              total        used        free      shared  buff/cache   available  
Mem:           15Gi        146Mi        14Gi         0.0Ki        275Mi        14Gi  
Swap:           0B           0B           0B  
[ec2-user@ip-172-31-4-6 ~]$
```

Now generating memory heavy load like redis generate using stress-ng to check the instance working and when i see the monitoring for the instance the cpu utilisation spikes suddenly indicating heavy workload but the performance is not affected much .



```
ec2-user@ip-172-31-4-6:~  
top - 10:47:16 up 27 min,  2 users,  load average: 0.34, 0.36, 0.19  
Tasks: 142 total,  2 running, 140 sleeping,   0 stopped,   0 zombie  
%Cpu(s): 21.4 us, 28.7 sy,  0.0 ni, 49.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st  
MiB Mem : 15623.6 total, 13784.8 free,  1479.8 used,   359.0 buff/cache  
MiB Swap:   0.0 total,   0.0 free,   0.0 used. 13877.5 avail Mem  
  
  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND  
 26850 ec2-user  20   0 8421384  1.3g 1040 R 100.0   8.5   0:14.38 stress-+  
 26851 ec2-user  20   0 224016   3500 2840 R   0.7   0.0   0:00.02 top  
  1396 root      20   0  53540 15232 14104 S   0.3   0.1   0:00.15 systemd+  
    1 root      20   0 172412 17404 10844 S   0.0   0.1   0:00.89 systemd  
    2 root      20   0     0     0     0 S   0.0   0.0   0:00.00 kthreadd  
    3 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 rcu_gp  
    4 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 rcu_par+  
    5 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 slub_fl+  
    6 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 netns  
    8 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 kworker+  
   10 root       0 -20     0     0     0 I   0.0   0.0   0:00.00 mm_perc+  
   11 root      20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tas+  
   12 root      20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tas+  
   13 root      20   0     0     0     0 I   0.0   0.0   0:00.00 rcu_tas+  
   14 root      20   0     0     0     0 S   0.0   0.0   0:00.04 ksoftir+  
   15 root      20   0     0     0     0 I   0.0   0.0   0:00.03 rcu_pre+  
   16 root      rt   0     0     0     0 S   0.0   0.0   0:00.00 migrati+
```

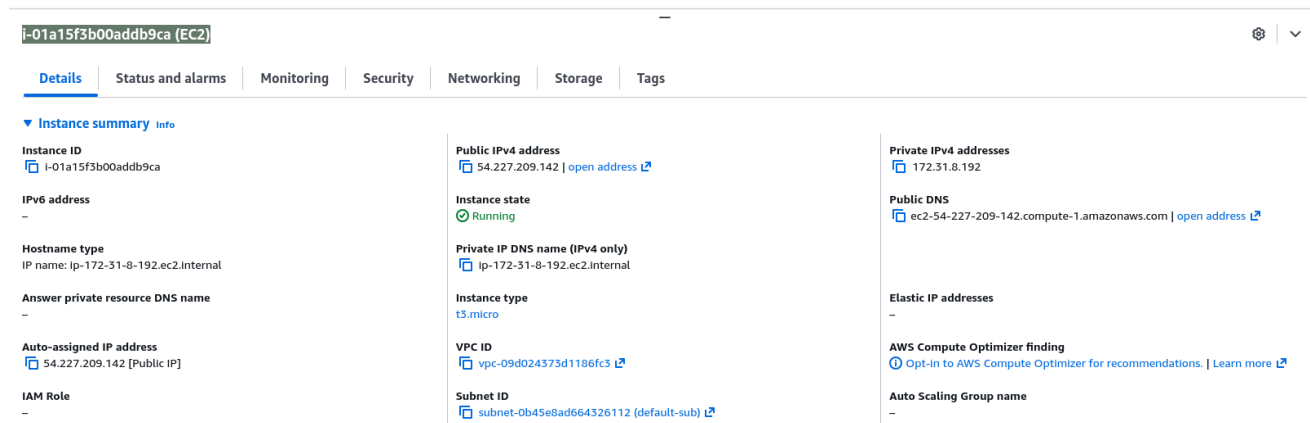
The top command shows that the stress process occupies the CPU and memory completely

but the instance can still handle it correctly. So my chosen instance work as expected and can handle extensive memory workload like redis would do or generate.

Q3. Launch an EC2 instance with an auto-assigned public IP, SSH into it and record its private and public IP from inside the instance, then stop and start it to observe how the IPs change; after that, allocate and attach an Elastic IP so that across subsequent stop/start cycles the externally reachable address remains stable, and verify the behavior.

Ans:

The EC2 instance with auto assigned public IP :



Instance summary

Property	Value
Instance ID	i-01a15f3b00adb9ca
IPv6 address	-
Hostname type	IP name: ip-172-31-8-192.ec2.internal
Answer private resource DNS name	-
Auto-assigned IP address	54.227.209.142 [Public IP]
IAM Role	-
Public IPv4 address	54.227.209.142 [open address]
Instance state	Running
Private IP DNS name (IPv4 only)	ip-172-31-8-192.ec2.internal
Instance type	t3.micro
VPC ID	vpc-09d024373d1186fc3
Subnet ID	subnet-0b45e8ad664326112 (default-sub)
Private IPv4 addresses	172.31.8.192
Public DNS	ec2-54-227-209-142.compute-1.amazonaws.com [open address]
Elastic IP addresses	-
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendations. [Learn more]
Auto Scaling Group name	-

Now login into the instance

```
`curl -H "X-aws-ec2-metadata-token: $TOKEN"
```

<http://169.254.169.254/latest/meta-data/public-ipv4>

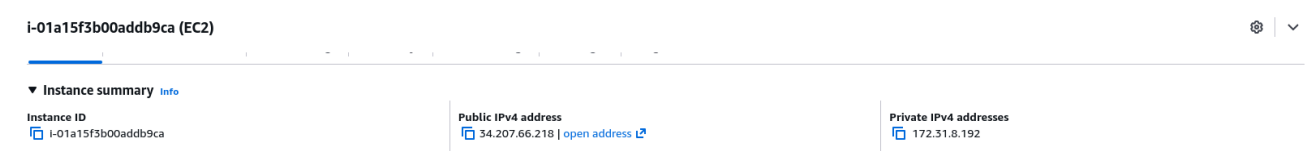
This shows the public IP used by ec2 instance.

```
curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/local-ipv4
```

This shows the private IP used by ec2 instance.

```
[ec2-user@ip-172-31-8-192 ~]$ curl -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4
54.227.209.142[ec2-user@ip-172-31-8-192 ~]$ curl -H "X-aws-ec2-metadata-token: $
TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4
172.31.8.192[ec2-user@ip-172-31-8-192 ~]$
```

Now I stopped my ec2 instance and then restarted it . The public IP changed .



Instance summary

Property	Value
Instance ID	i-01a15f3b00adb9ca
Public IPv4 address	34.207.66.218 [open address]
Private IPv4 addresses	172.31.8.192

Elastic IP address associated successfully.
Elastic IP address 98.94.138.195 has been associated with Instance I-01a15f3b00adb9ca

Elastic IP addresses (1) Info

Public IPv4 address: 98.94.138.195

Clear Filters

☐	Name	Allocated IPv4 address	Type	Allocation ID	Reverse DNS record	Associated instance ID	Private IP address
☐	-	98.94.138.195	Public IP	elpallo-026bdfa804373b037	-	I-01a15f3b00adb9ca	172.31.8.192

```

AQAEAGmxuFBW4eVMSEKkFCIUrhIfC-bl_h644HQLum_bfQtK5UuZg==
[ec2-user@ip-172-31-8-192 ~]$ curl -H "X-aws-ec2-metadata-token: $TOKEN" http://
169.254.169.254/latest/meta-data/public-ipv4
98.94.138.195[ec2-user@ip-172-31-8-192 ~]$ curl -H "X-aws-ec2-metadata-token: $
TOKEN" http://169.254.169.254/latest/meta-data/local-ipv4
172.31.8.192[ec2-user@ip-172-31-8-192 ~]$

```

[illegible]

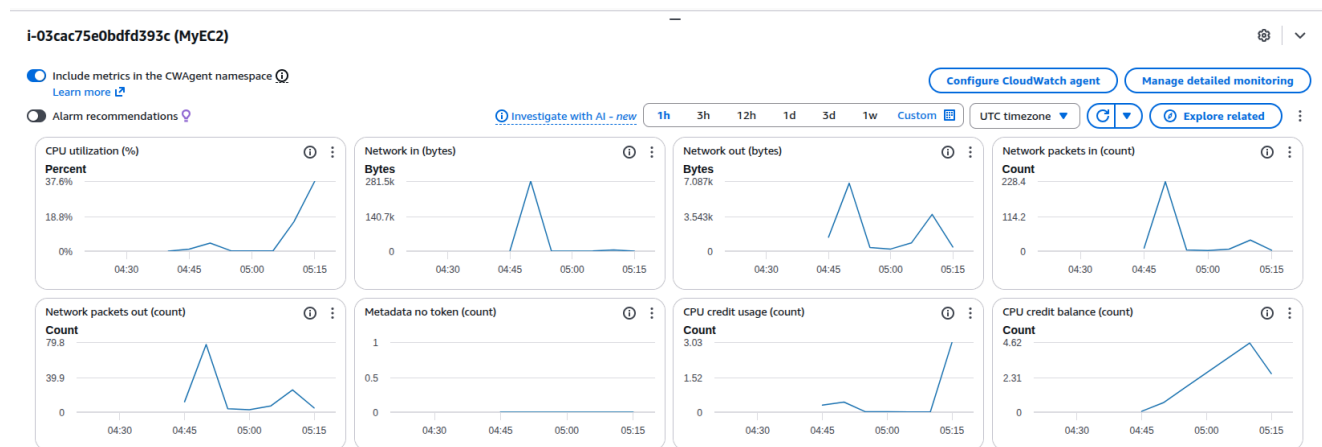
Q4. Simulate losing the SSH private key for a Linux EC2 instance by “forgetting” it, then regain access without recreating the instance by using a second rescue instance: detach the original instance’s root volume, attach it to the rescue instance, fix SSH access (for example by updating `authorized_keys`), reattach the volume to the original instance, and confirm you can log in again.

First I have created my EC2 instance , this has access with key - original -key with which I will simulate the losing of the SSH key . So now I have my key so normal ec2 login :

Q5. Launch an EC2 instance, generate sustained CPU load and some network traffic from it, then use CloudWatch to view CPU Utilization, NetworkIn, and NetworkOut metrics over at least 10–15 minutes, correlating specific actions you took on the instance with visible patterns on the graphs.

Ans :

Launched the EC2 with t3.micro instance and generated sustained CPU load using stress tool



The EC2 instance under the stress and monitoring shows the expected behaviour and sudden spike in network in and network out and CPU utilisation.

Q6. Launch an EC2 instance with a user data script intended to install Nginx and serve a custom “Hello” page, deliberately introduce a bug in the script so that it fails, then debug the failure using cloud-init and other relevant logs, correct the script, and relaunch or re-run it until the instance comes up with the page working.

Ans:

I have launched my EC2 instance with a buggy user data script which could'nt install the nginx .

```
+-----[SHA256]-----+
Cloud-init v. 22.2.2 running 'modules:config' at Fri, 30 Jan 2026 05:39:28 +0000
. Up 7.80 seconds.
Cloud-init v. 22.2.2 running 'modules:final' at Fri, 30 Jan 2026 05:39:28 +0000.
Up 8.45 seconds.
Amazon Linux 2023 repository                61 MB/s |  53 MB      00:00
Amazon Linux 2023 Kernel Livepatch repository 244 kB/s |  30 kB      00:00
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:02 ago on Fri Jan 30 05:39:42 2026.
No match for argument: nginx
Error: Unable to find a match: nginx
Failed to start nginx.service: Unit nginx.service not found.
Failed to enable unit: Unit file nginx.service does not exist.
/var/lib/cloud/instance/scripts/part-001: line 7: /usr/share/nginx/html/index.ht
ml: No such file or directory
2026-01-30 05:39:44,521 - cc_scripts_user.py[WARNING]: Failed to run module scri
pts-user (scripts in /var/lib/cloud/instance/scripts)
2026-01-30 05:39:44,522 - util.py[WARNING]: Running module scripts-user (<module
'cloudinit.config.cc_scripts_user' from '/usr/lib/python3.9/site-packages/cloud
init/config/cc_scripts_user.py'>) failed
Cloud-init v. 22.2.2 finished at Fri, 30 Jan 2026 05:39:44 +0000. Datasource Dat
aSourceEc2. Up 24.25 seconds
[ec2-user@ip-172-31-13-155 ~]$
```

```
[ec2-user@ip-172-31-13-155 ~]$ sudo systemctl nginx
Unknown command verb nginx.
[ec2-user@ip-172-31-13-155 ~]$ sudo systemctl status nginx
Unit nginx.service could not be found.
[ec2-user@ip-172-31-13-155 ~]$
```

Now correcting the script from inside the ec2 instance by going into the /var/lib/ directory.

```
ec2-user@ip-172-31-13-155:/var/lib/cloud/instance/scripts
[ec2-user@ip-172-31-13-155 scripts]$ cat part-001
cat: part-001: Permission denied
[ec2-user@ip-172-31-13-155 scripts]$ ls -al
total 20
drwxr-xr-x. 2 root root    22 Jan 30 05:39 .
drwxr-xr-x. 5 root root 16384 Jan 30 05:39 ..
-rwx----- 1 root root   162 Jan 30 05:39 part-001
[ec2-user@ip-172-31-13-155 scripts]$ sudo cat part-001
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

echo "<h1>Hello from EC2</h1>" > /usr/share/nginx/html/index.html
[ec2-user@ip-172-31-13-155 scripts]$ sudo vi part-001
[ec2-user@ip-172-31-13-155 scripts]$ sudo cat part-001
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx

echo "<h1>Hello from EC2</h1>" > /usr/share/nginx/html/index.html
[ec2-user@ip-172-31-13-155 scripts]$
```

Now the script ran correctly and nginx worked.

```
[ec2-user@ip-172-31-13-155 scripts]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-01-30 05:52:17 UTC; 29s ago
     Main PID: 3406 (nginx)
        Tasks: 3 (limit: 1067)
       Memory: 3.2M
          CPU: 56ms
      CGroup: /system.slice/nginx.service
              └─3406 "nginx: master process /usr/sbin/nginx"
                 └─3408 "nginx: worker process"
                    └─3409 "nginx: worker process"

Jan 30 05:52:16 ip-172-31-13-155.ec2.internal systemd[1]: Starting nginx.service:
Jan 30 05:52:17 ip-172-31-13-155.ec2.internal nginx[3305]: nginx: the configura
Jan 30 05:52:17 ip-172-31-13-155.ec2.internal nginx[3305]: nginx: configura
Jan 30 05:52:17 ip-172-31-13-155.ec2.internal systemd[1]: Started nginx.servic
lines 1-16/16 (END)
```

Q7. Create an 8 GB gp3 EBS volume, attach it to a running instance, format and mount it (for example at /data), make the mount persistent using /etc/fstab , then intentionally introduce an error in fstab that prevents normal boot and recover the instance without data loss by fixing fstab via a rescue instance and reattaching the volume.

Ans :

Created my ec2 instance and attached the created volume and then formatted it and mounted it at /data directory.

```
ec2-user@ip-172-31-14-0:~$ sudo mkfs -t ext4 /dev/nvme1n1
Proceed anyway? (y,N) y
Creating filesystem with 2097152 4k blocks and 524288 inodes
Filesystem UUID: 619a1f44-a650-46e8-b300-58eb72986511
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

[ec2-user@ip-172-31-14-0 ~]$ sudo mkdir /data
[ec2-user@ip-172-31-14-0 ~]$ sudo mount /dev/nvme1n1 /data
[ec2-user@ip-172-31-14-0 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           459M   0    459M   0% /dev/shm
tmpfs           184M  440K   183M   1% /run
/dev/nvme0n1p1   8.0G  1.6G   6.5G  19% /
tmpfs           459M   0    459M   0% /tmp
/dev/nvme0n1p128 10M   1.3M   8.7M  13% /boot/efi
tmpfs           92M    0     92M   0% /run/user/1000
/dev/nvme1n1     7.8G  24K   7.4G   1% /data
[ec2-user@ip-172-31-14-0 ~]$
```

Now intentionally introducing the error in fstab. The SSH connection refused because the instance entered in boot failure.

```
piyush@CK-piyush:~$ df -h
tmpfs           184M  440K   183M   1% /run
/dev/nvme0n1p1   8.0G  1.6G   6.5G  19% /
tmpfs           459M   0    459M   0% /tmp
/dev/nvme0n1p128 10M   1.3M   8.7M  13% /boot/efi
tmpfs           92M    0     92M   0% /run/user/1000
/dev/nvme1n1     7.8G  24K   7.4G   1% /data
[ec2-user@ip-172-31-14-0 ~]$ sudo blkid /dev/nvme1n1
/dev/nvme1n1: UUID="619a1f44-a650-46e8-b300-58eb72986511" BLOCK_SIZE="4096" TYPE="ext4"
[ec2-user@ip-172-31-14-0 ~]$ sudo nano /etc/fstab
[ec2-user@ip-172-31-14-0 ~]$ ^C
[ec2-user@ip-172-31-14-0 ~]$ sudo nano /etc/fstab
[ec2-user@ip-172-31-14-0 ~]$ sudo umount /data
[ec2-user@ip-172-31-14-0 ~]$ sudo mount -a
[ec2-user@ip-172-31-14-0 ~]$ sudo nano /etc/fstab
[ec2-user@ip-172-31-14-0 ~]$ sudo reboot

Broadcast message from root@localhost on pts/1 (Fri 2026-01-30 07:14:40 UTC):

The system will reboot now!

Connection to 54.152.249.22 closed by remote host.
Connection to 54.152.249.22 closed.
piyush@CK-piyush:~$
```

Now recovering the instance using rescue instance

```

+ ec2-user@ip-172-31-8-145:~
└─nvme1n1p128 259:7 0 10M 0 part
[ec2-user@ip-172-31-8-145 ~]$ sudo nano /recovery/etc/fstab
[ec2-user@ip-172-31-8-145 ~]$ sudo blkid
/dev/nvme0n1p1: LABEL="/" UUID="298c4de6-f9e5-428a-96bf-cc54c4bf3079" BLOCK_SIZE
="4096" TYPE="xfs" PARTLABEL="Linux" PARTUUID="56b4ac6e-ae91-4305-b021-14727ea21
5e7"
/dev/nvme0n1p128: SEC_TYPE="msdos" UUID="8FB3-6A0A" BLOCK_SIZE="512" TYPE="vfat"
PARTLABEL="EFI System Partition" PARTUUID="bf96014f-483c-4ea8-ac03-7e816df8d345
"
/dev/nvme0n1p127: PARTLABEL="BIOS Boot Partition" PARTUUID="758f5268-da23-4301-b
a69-169f71f5202b"
/dev/nvme1n1p127: PARTLABEL="BIOS Boot Partition" PARTUUID="758f5268-da23-4301-b
a69-169f71f5202b"
/dev/nvme1n1p1: LABEL="/" UUID="298c4de6-f9e5-428a-96bf-cc54c4bf3079" BLOCK_SIZE
="4096" TYPE="xfs" PARTLABEL="Linux" PARTUUID="56b4ac6e-ae91-4305-b021-14727ea21
5e7"
/dev/nvme1n1p128: SEC_TYPE="msdos" UUID="8FB3-6A0A" BLOCK_SIZE="512" TYPE="vfat"
PARTLABEL="EFI System Partition" PARTUUID="bf96014f-483c-4ea8-ac03-7e816df8d345
"
[ec2-user@ip-172-31-8-145 ~]$ sudo nano /recovery/etc/fstab
[ec2-user@ip-172-31-8-145 ~]$ lsblk -dno UUID /dev/nvme1n1p1
298c4de6-f9e5-428a-96bf-cc54c4bf3079
[ec2-user@ip-172-31-8-145 ~]$ sudo nano /recovery/etc/fstab
[ec2-user@ip-172-31-8-145 ~]$

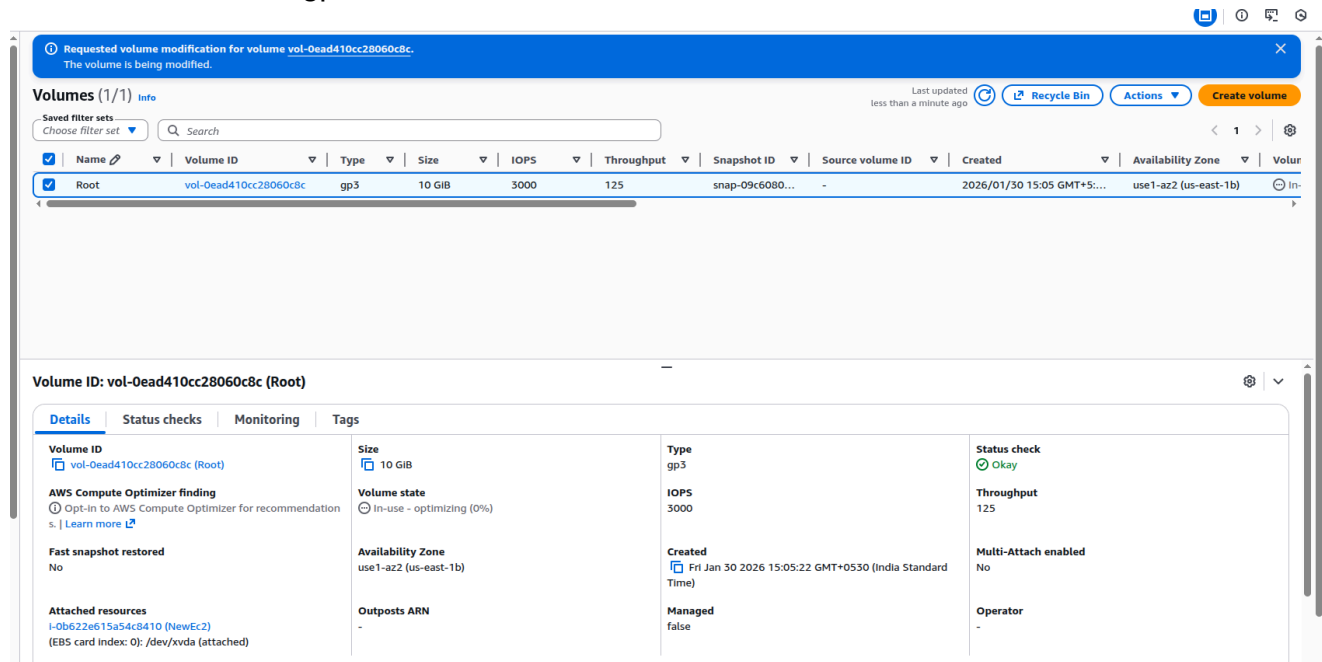
```

```
+ ec2-user@ip-172-31-14-96:~  
This host key is known by the following other names/addresses:  
~/ssh/known_hosts:54: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '18.205.243.114' (ED25519) to the list of known hosts  
. , #_  
~\ _ #####_ Amazon Linux 2023  
~~ \#####\  
~~ \###|  
~~ \#/ _____ https://aws.amazon.com/linux/amazon-linux-2023  
~~ V~' '->  
~~~~  
~~ . - . _/  
_/_/_/  
_/m/'  
Last login: Fri Jan 30 08:58:21 2026 from 106.192.193.68  
[ec2-user@ip-172-31-14-96 ~]$ lsblk  
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS  
nvme1n1 259:0 0 8G 0 disk  
nvme0n1 259:1 0 8G 0 disk  
└─nvme0n1p1 259:2 0 8G 0 part /  
└─nvme0n1p127 259:3 0 1M 0 part  
└─nvme0n1p128 259:4 0 10M 0 part /boot/efi  
[ec2-user@ip-172-31-14-96 ~]$
```


grow the partition/filesystem so the operating system recognizes and uses the full 10 GB, confirming both the volume and filesystem resize paths.

Ans:

I have created my ec2 instance initially with 8GB gp3 volume and then I modified it increase the volume to 10GB gp3.



The screenshot shows the AWS Management Console interface for an EBS volume. At the top, a blue banner indicates a requested volume modification for volume `vol-0ead410cc28060c8c`, stating "The volume is being modified." Below this, the "Volumes (1/1)" section displays a table with the following data:

Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Source volume ID	Created	Availability Zone
Root	vol-0ead410cc28060c8c	gp3	10 GiB	3000	125	snap-09c6080...	-	2026/01/30 15:05 GMT+5:...	use1-az2 (us-east-1b)

Below the table, the "Volume ID: vol-0ead410cc28060c8c (Root)" details are shown. The "Details" tab is active, displaying the following information:

- Volume ID:** vol-0ead410cc28060c8c (Root)
- Size:** 10 GiB
- Type:** gp3
- Status check:** ✔ Okay
- AWS Compute Optimizer finding:** Opt-In to AWS Compute Optimizer for recommendation. [Learn more](#)
- Volume state:** In-use - optimizing (0%)
- IOPS:** 3000
- Throughput:** 125
- Fast snapshot restored:** No
- Availability Zone:** use1-az2 (us-east-1b)
- Created:** Fri Jan 30 2026 15:05:22 GMT+0530 (India Standard Time)
- Multi-Attach enabled:** No
- Attached resources:** i-0b622e615a54c8410 (NewEc2) (EBS card index: 0: /dev/xvda (attached))
- Outposts ARN:** -
- Managed:** false
- Operator:** -

```
ec2-user@ip-172-31-13-255:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1      259:0    0  10G  0 disk
├─nvme0n1p1  259:1    0   8G  0 part /
├─nvme0n1p127 259:2    0   1M  0 part
└─nvme0n1p128 259:3    0  10M  0 part /boot/efi
ec2-user@ip-172-31-13-255:~$
```

The using `sudo growpart /dev/nvme0n1 1`
I created the partition .


```

Last login: Fri Jan 30 09:39:25 2026 from 106.192.193.68
[ec2-user@ip-172-31-13-255 ~]$ lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1              259:0    0   10G  0 disk
├─nvme0n1p1          259:1    0    8G  0 part /
├─nvme0n1p127        259:2    0    1M  0 part
└─nvme0n1p128        259:3    0   10M  0 part /boot/efi
[ec2-user@ip-172-31-13-255 ~]$ sudo growpart /dev/nvme0n1 1
CHANGED: partition=1 start=24576 old: size=16752607 end=16777183 new: size=20946911 end=20971487
[ec2-user@ip-172-31-13-255 ~]$ lsblk
NAME                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1              259:0    0   10G  0 disk
├─nvme0n1p1          259:1    0   10G  0 part /
├─nvme0n1p127        259:2    0    1M  0 part
└─nvme0n1p128        259:3    0   10M  0 part /boot/efi
[ec2-user@ip-172-31-13-255 ~]$

```

Final check with `df -h` shows that now the instance has 10GB space to use.

Commands used : `sudo xfs_growfs -d /`

```

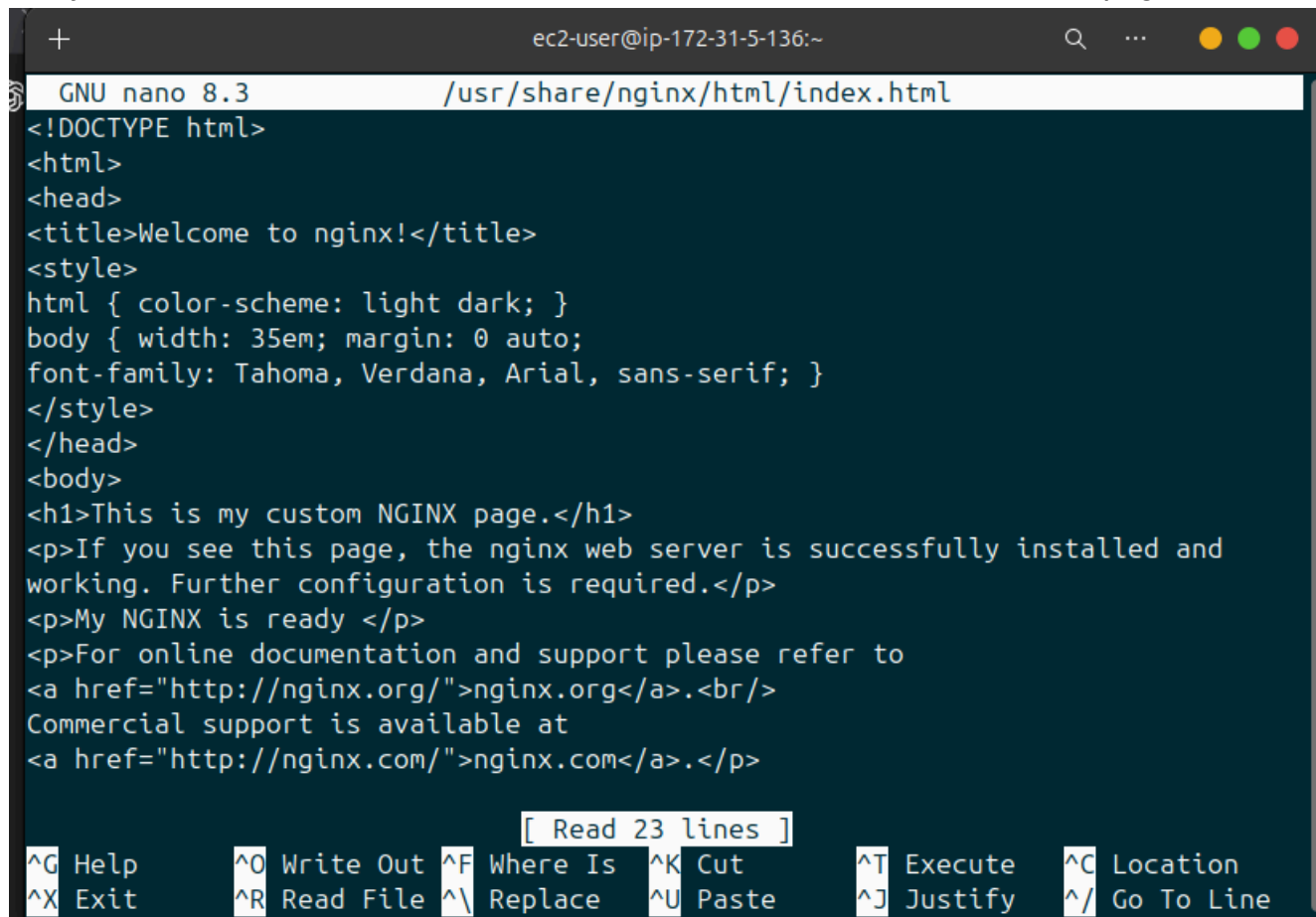
+ ec2-user@ip-172-31-13-255:~
tmpfs                tmpfs                93884            0            93884        0% /run/user/1000
[ec2-user@ip-172-31-13-255 ~]$ sudo xfs_growfs -d /
meta-data=/dev/nvme0n1p1      isize=512    agcount=2, agsize=1047040 blks
      =                       sectsz=4096   attr=2, projid32bit=1
      =                       crc=1          finobt=1, sparse=0, rmapbt=0
      =                       reflink=0       bigtime=1 inobtcount=1 nrext64=0
      =                       exchange=0
data      =                   bsize=4096   blocks=2094075, imaxpct=25
      =                   sunit=128    swidth=128 blks
naming    =version 2          bsize=16384  ascii-ci=0, ftype=1, parent=0
log       =internal log      bsize=4096   blocks=16384, version=2
      =                   sectsz=4096   sunit=4 blks, lazy-count=1
realtime  =none              extsz=4096   blocks=0, rtextents=0
data blocks changed from 2094075 to 2618363
[ec2-user@ip-172-31-13-255 ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        4.0M   0    4.0M   0% /dev
tmpfs           459M   0    459M   0% /dev/shm
tmpfs           184M 440K  183M   1% /run
/dev/nvme0n1p1  10G  1.6G   8.5G  16% /
tmpfs           459M   0    459M   0% /tmp
/dev/nvme0n1p128 10M  1.3M   8.7M  13% /boot/efi
tmpfs           92M   0     92M   0% /run/user/1000
[ec2-user@ip-172-31-13-255 ~]$

```

Q9. Take an EC2 instance where Nginx is already installed with a custom index page, identify its root EBS volume, create a snapshot of that volume, and from this snapshot create an AMI such that any new instance launched from this AMI boots with Nginx preinstalled and serving the same custom page without additional manual setup.

Ans:

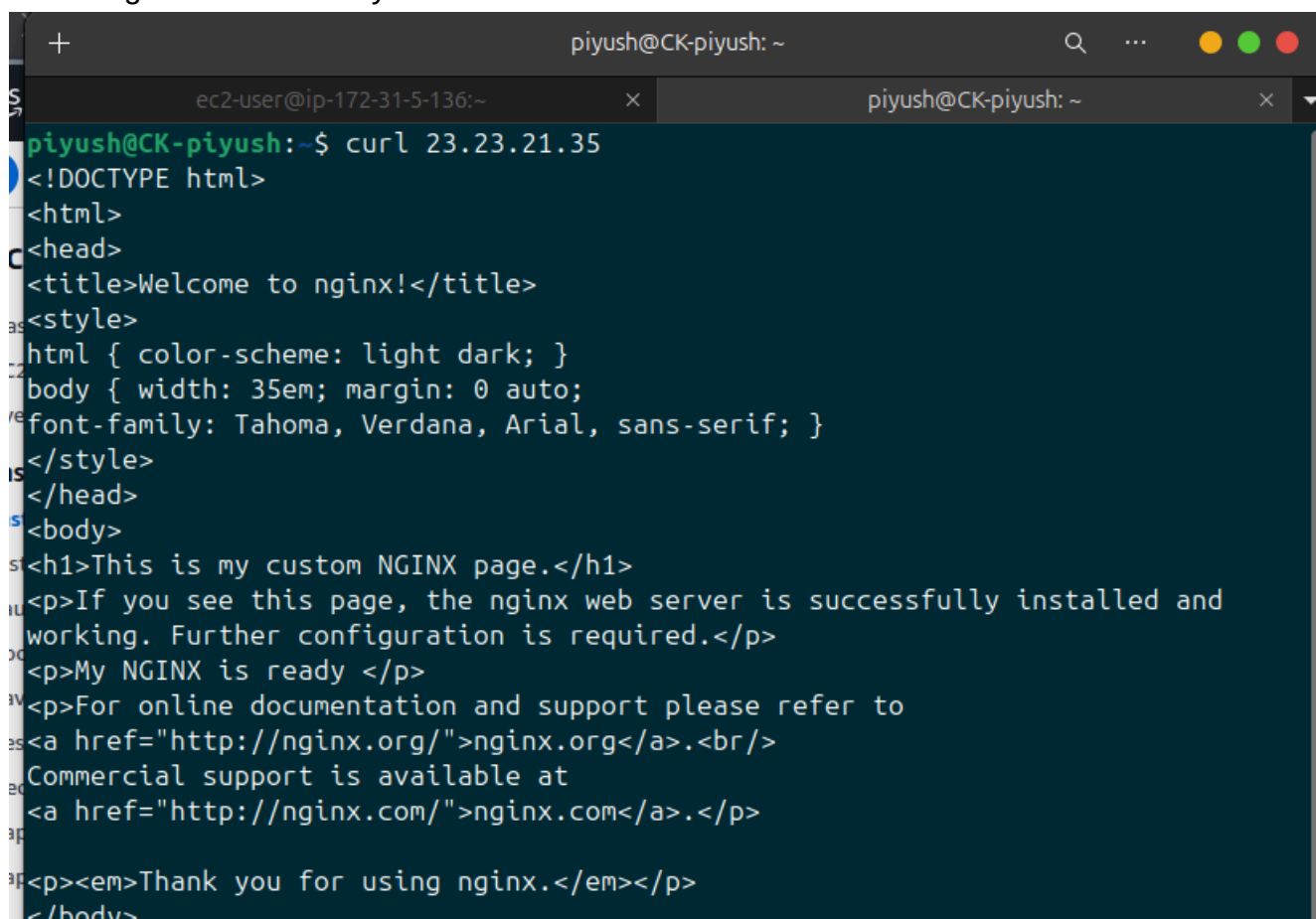
In my EC2 instance , I installed NGINX and then created a custom index.html page.



```
ec2-user@ip-172-31-5-136:~
GNU nano 8.3 /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>This is my custom NGINX page.</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
<p>My NGINX is ready </p>
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

[ Read 23 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Checking the server on my terminal



```
piyush@CK-piyush: ~
ec2-user@ip-172-31-5-136:~ x
piyush@CK-piyush: ~ x
piyush@CK-piyush:~$ curl 23.23.21.35
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>This is my custom NGINX page.</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
<p>My NGINX is ready </p>
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>
<p><em>Thank you for using nginx.</em></p>
</body>
```

The AMI is created from this ec2 instance successfully.

```
ec2-user@ip-172-31-12-112:~  
_/_/_/  
_/_/_/  
Last login: Fri Jan 30 15:06:23 2026 from 103.83.148.66  
[ec2-user@ip-172-31-12-112 ~]$ sudo systemctl status nginx  
● nginx.service - The nginx HTTP and reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)  
   Active: active (running) since Fri 2026-01-30 15:19:43 UTC; 31s ago  
     Process: 1556 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)  
     Process: 1561 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)  
     Process: 1593 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)  
    Main PID: 1597 (nginx)  
       Tasks: 3 (limit: 1067)  
      Memory: 6.9M  
         CPU: 54ms  
      CGroup: /system.slice/nginx.service  
              └─1597 "nginx: master process /usr/sbin/nginx"  
                  └─1598 "nginx: worker process"  
                      └─1599 "nginx: worker process"  
  
Jan 30 15:19:43 ip-172-31-12-112.ec2.internal systemd[1]: Starting nginx.service:  
Jan 30 15:19:43 ip-172-31-12-112.ec2.internal nginx[1561]: nginx: the configura  
Jan 30 15:19:43 ip-172-31-12-112.ec2.internal nginx[1561]: nginx: configura  
Jan 30 15:19:43 ip-172-31-12-112.ec2.internal systemd[1]: Started nginx.service
```

Now checking the custom page. It works fine .

```
piyush@CK-piyush: ~  
ec2-user@ip-172-31-12-112:~ x  
piyush@CK-piyush: ~ x  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to nginx!</title>  
<style>  
html { color-scheme: light dark; }  
body { width: 35em; margin: 0 auto;  
font-family: Tahoma, Verdana, Arial, sans-serif; }  
</style>  
</head>  
<body>  
<h1>This is my custom NGINX page.</h1>  
<p>If you see this page, the nginx web server is successfully installed and  
working. Further configuration is required.</p>  
<p>My NGINX is ready </p>  
<p>For online documentation and support please refer to  
<a href="http://nginx.org/">nginx.org</a>.<br/>  
Commercial support is available at  
<a href="http://nginx.com/">nginx.com</a>.</p>  
  
<p><em>Thank you for using nginx.</em></p>  
</body>  
</html>
```

Q10. Create a new encrypted gp3 EBS volume using a KMS key, attach it to an instance and use it for some real data, then take a snapshot and create another volume from that

snapshot, observing how encryption is handled throughout and verifying that data is readable as normal inside the instance while remaining encrypted at rest in AWS

Ans:

I created an encrypted gp3 EBS volume using a default aws/ebs KMS key.

Volume ID: vol-0970e37d4cdba1cd1 (Encrypted-Volume)			
Details	Status checks	Monitoring	Tags
Volume ID vol-0970e37d4cdba1cd1 (Encrypted-Volume)	Size 8 GiB	Type gp3	Status check Okay
AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more	Volume state Available	IOPS 3000	Throughput 125
Fast snapshot restored No	Availability Zone us-east-2 (us-east-1b)	Created Fri Jan 30 2026 22:27:53 GMT+0530 (India Standard Time)	Multi-Attach enabled No
Attached resources -	Outposts ARN -	Managed false	Operator -
Source			
Snapshot ID -	Source volume ID -		
Encryption			
Encryption Encrypted	KMS key ID c649fae9-215b-40e8-91b2-b50e54a108d6	KMS key alias aws/ebs	KMS key ARN arn:aws:kms:us-east-1:797265029671:key/c649fae9-215b-40e8-91b2-b50e54a108d6

Using some testing data to show working for encryption .

```
ec2-user@ip-172-31-5-136:~  
Writing superblocks and filesystem accounting information: done  
  
[ec2-user@ip-172-31-5-136 ~]$ sudo mkdir /secure-data  
[ec2-user@ip-172-31-5-136 ~]$ sudo mount /dev/nvme1n1 /secure-data  
[ec2-user@ip-172-31-5-136 ~]$ df -h  
Filesystem      Size  Used Avail Use% Mounted on  
devtmpfs        4.0M   0    4.0M   0% /dev  
tmpfs           459M   0    459M   0% /dev/shm  
tmpfs           184M  440K   183M   1% /run  
/dev/nvme0n1p1  8.0G  1.6G   6.4G  20% /  
tmpfs           459M   0    459M   0% /tmp  
/dev/nvme0n1p128 10M   1.3M   8.7M  13% /boot/efi  
tmpfs           92M    0     92M   0% /run/user/1000  
/dev/nvme1n1    7.8G   24K   7.4G   1% /secure-data  
[ec2-user@ip-172-31-5-136 ~]$ sudo tee /secure-data/secret.txt <<EOF  
> This is sensitive production data.  
> Stored on encrypted EBS  
> EOF  
This is sensitive production data.  
Stored on encrypted EBS  
[ec2-user@ip-172-31-5-136 ~]$ cat /secure-data/secret.txt  
This is sensitive production data.  
Stored on encrypted EBS  
[ec2-user@ip-172-31-5-136 ~]$
```

The snapshot created from the encrypted volume.

Snapshot ID: snap-040d23fd99e3a4ccf (Snapshot)

Details

Snapshot settings

Storage tier

Tags

Snapshot ID

snap-040d23fd99e3a4ccf (Snapshot)

Owner

797265029671

Description

This is a snapshot from encrypted volume.

▼ Source volume

Volume ID

vol-0970e37d4cdba1cd1

▼ Encryption

Encryption

Encrypted

Full snapshot size

-

Started

Fri Jan 30 2026 22:37:26 GMT+0530 (India Standard Time)

Progress

0%

Product codes

-

Snapshot status

Pending

Fast snapshot restore

-

KMS key ID

c649fae9-215b-40e8-91b2-b50e54a108d6

KMS key alias

aws/ebs

KMS key ARN

arn:aws:kms:us-east-1:797265029671:key/c649fae9-215b-40e8-91b2-b50e54a108d6

Now the volume is created from the snapshot which is encrypted .

Volume ID: vol-09449ee20b0a2282a (Snap-Volume)

Details

Status checks

Monitoring

Tags

Volume ID

vol-09449ee20b0a2282a (Snap-Volume)

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Fast snapshot restored

No

Attached resources

-

▼ Source

Snapshot ID

snap-040d23fd99e3a4ccf

▼ Encryption

Encryption

Encrypted

Size

8 GiB

Volume state

Available

Availability Zone

use1-az2 (us-east-1b)

Outposts ARN

-

Type

gp3

IOPS

3000

Created

Fri Jan 30 2026 22:39:53 GMT+0530 (India Standard Time)

Managed

false

Status check

Okay

Throughput

125

Multi-Attach enabled

No

Operator

-

KMS key ID

c649fae9-215b-40e8-91b2-b50e54a108d6

KMS key alias

aws/ebs

KMS key ARN

arn:aws:kms:us-east-1:797265029671:key/c649fae9-215b-40e8-91b2-b50e54a108d6

Now I attached the snapshotted volume to my current instance and then mounted the partition on my instance and then checked that my encrypted data was available.

```

+ ec2-user@ip-172-31-5-136:~
#_
~\_ #####_ Amazon Linux 2023
~~ \#####\
~~ \|###|
~~ \|#/ https://aws.amazon.com/linux/amazon-linux-2023
~~ V~' '->
~~~ /
~~._.-/_/_/
~~/_/_/_/
~~/_m/'

Last login: Fri Jan 30 17:00:15 2026 from 27.59.76.3
[ec2-user@ip-172-31-5-136 ~]$ lsblk
NAME            MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1          259:0    0   8G  0 disk
├─nvme0n1p1       259:1    0   8G  0 part /
├─nvme0n1p127     259:2    0   1M  0 part
└─nvme0n1p128     259:3    0  10M  0 part /boot/efi
nvme1n1          259:4    0   8G  0 disk /secure-data
nvme2n1          259:5    0   8G  0 disk
[ec2-user@ip-172-31-5-136 ~]$ sudo mkdir /securedata-copy
[ec2-user@ip-172-31-5-136 ~]$ sudo mount /dev/nvme2n1 /securedata-copy
[ec2-user@ip-172-31-5-136 ~]$ cat /securedata-copy/secret.txt
This is sensitive production data.
Stored on encrypted EBS
[ec2-user@ip-172-31-5-136 ~]$
```